

Business Problem

DataCo ships online orders around the world. It can only afford to improve delivery in three regions this year. Pick the three regions where late deliveries are costing the most business.

```
In [1]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [2]: df = pd.read_csv(r'C:\Users\shobh\Downloads\SupplyChainDataset\DataCoSupplyChainDat
```

```
In [3]: #pd.set_option('display.max_columns', None)
df.head()
```

Out[3]:

	Type	Days for shipping (real)	Days for shipment (scheduled)	Benefit per order	Sales per customer	Delivery Status	Late_delivery_risk
0	DEBIT	3	4	91.250000	314.640015	Advance shipping	0
1	TRANSFER	5	4	-249.089996	311.359985	Late delivery	1
2	CASH	4	4	-247.779999	309.720001	Shipping on time	0
3	DEBIT	3	4	22.860001	304.809998	Advance shipping	0
4	PAYMENT	2	4	134.210007	298.250000	Advance shipping	0

5 rows × 53 columns



```
In [4]: # Checking data columns names

df.columns
```

```
Out[4]: Index(['Type', 'Days for shipping (real)', 'Days for shipment (scheduled)',
              'Benefit per order', 'Sales per customer', 'Delivery Status',
              'Late_delivery_risk', 'Category Id', 'Category Name', 'Customer City',
              'Customer Country', 'Customer Email', 'Customer Fname', 'Customer Id',
              'Customer Lname', 'Customer Password', 'Customer Segment',
              'Customer State', 'Customer Street', 'Customer Zipcode',
              'Department Id', 'Department Name', 'Latitude', 'Longitude', 'Market',
              'Order City', 'Order Country', 'Order Customer Id',
              'order date (DateOrders)', 'Order Id', 'Order Item Cardprod Id',
              'Order Item Discount', 'Order Item Discount Rate', 'Order Item Id',
              'Order Item Product Price', 'Order Item Profit Ratio',
              'Order Item Quantity', 'Sales', 'Order Item Total',
              'Order Profit Per Order', 'Order Region', 'Order State', 'Order Status',
              'Order Zipcode', 'Product Card Id', 'Product Category Id',
              'Product Description', 'Product Image', 'Product Name', 'Product Price',
              'Product Status', 'shipping date (DateOrders)', 'Shipping Mode'],
              dtype='str')
```

```
In [5]: df.shape
```

```
Out[5]: (180519, 53)
```

Eliminating the columns not needed for Analysis

```
In [6]: # based on the objective of the analysis, we are eliminating the columns that are n

df =df.drop(columns=['Benefit per order','Sales per customer','Late_delivery_risk',
                    'Customer Lname', 'Customer Password', 'Customer Segment',
                    'Customer State', 'Customer Street', 'Customer Zipcode', 'Department Id', 'L
                    'Order Item Cardprod Id','Order Item Discount', 'Order State',
                    'Order Zipcode', 'Product Card Id', 'Product Category Id','Product Status',
```

```
In [7]: df
```

Out[7]:

	Type	Days for shipping (real)	Days for shipment (scheduled)	Delivery Status	Category Name	Customer City	Department Name	Me
0	DEBIT	3	4	Advance shipping	Sporting Goods	Caguas	Fitness	P
1	TRANSFER	5	4	Late delivery	Sporting Goods	Caguas	Fitness	P
2	CASH	4	4	Shipping on time	Sporting Goods	San Jose	Fitness	P
3	DEBIT	3	4	Advance shipping	Sporting Goods	Los Angeles	Fitness	P
4	PAYMENT	2	4	Advance shipping	Sporting Goods	Caguas	Fitness	P
...	...	...	...	...	...	...	...	
180514	CASH	4	4	Shipping on time	Fishing	Brooklyn	Fan Shop	P
180515	DEBIT	3	2	Late delivery	Fishing	Bakersfield	Fan Shop	P
180516	TRANSFER	5	4	Late delivery	Fishing	Bristol	Fan Shop	P
180517	PAYMENT	3	4	Advance shipping	Fishing	Caguas	Fan Shop	P
180518	PAYMENT	4	4	Shipping on time	Fishing	Caguas	Fan Shop	P
180519 rows × 26 columns								

In [8]: `# Checking data shape after column elimination`
`df.shape`

```
Out[8]: (180519, 26)
```

EDA Part 1 -

1. Column data type
2. Nulls values in each column
3. Checking data range
4. Duplicate rows
5. Eliminating Extra Spaces (column names or row values)
6. Checking for Outliers

```
In [9]: # Verifying data types for remaining columns
```

```
df.info()
```

```
<class 'pandas.DataFrame'>
```

```
RangeIndex: 180519 entries, 0 to 180518
```

```
Data columns (total 26 columns):
```

#	Column	Non-Null Count	Dtype
0	Type	180519 non-null	str
1	Days for shipping (real)	180519 non-null	int64
2	Days for shipment (scheduled)	180519 non-null	int64
3	Delivery Status	180519 non-null	str
4	Category Name	180519 non-null	str
5	Customer City	180519 non-null	str
6	Department Name	180519 non-null	str
7	Market	180519 non-null	str
8	Order City	180519 non-null	str
9	Order Country	180519 non-null	str
10	order date (DateOrders)	180519 non-null	str
11	Order Id	180519 non-null	int64
12	Order Item Discount Rate	180519 non-null	float64
13	Order Item Id	180519 non-null	int64
14	Order Item Product Price	180519 non-null	float64
15	Order Item Profit Ratio	180519 non-null	float64
16	Order Item Quantity	180519 non-null	int64
17	Sales	180519 non-null	float64
18	Order Item Total	180519 non-null	float64
19	Order Profit Per Order	180519 non-null	float64
20	Order Region	180519 non-null	str
21	Order Status	180519 non-null	str
22	Product Name	180519 non-null	str
23	Product Price	180519 non-null	float64
24	shipping date (DateOrders)	180519 non-null	str
25	Shipping Mode	180519 non-null	str

```
dtypes: float64(7), int64(5), str(14)
```

```
memory usage: 35.8 MB
```

Converting columns to correct data types

```
In [10]: #Converting Order date and shipping date columns to date column
```

```
cols=['order date (DateOrders)','shipping date (DateOrders)']  
df[cols] = df[cols].apply(pd.to_datetime)
```

```
In [11]: # changing Order Id, Order Item Id to Object from integer value
```

```
cols_string=['Order Id','Order Item Id']  
df[cols_string]=df[cols_string].astype('object')
```

Checking for data date range

```
In [12]: df[cols].min()
```

```
Out[12]: order date (DateOrders)      2015-01-01  
shipping date (DateOrders)      2015-01-03  
dtype: datetime64[us]
```

```
In [13]: df[cols].max()
```

```
Out[13]: order date (DateOrders)      2018-01-31 23:38:00  
shipping date (DateOrders)      2018-02-06 22:14:00  
dtype: datetime64[us]
```

Data range lies between - Jan 2015 - Jan 2018

(2018 data should be eliminated as we have data only for Jan which may skew the analysis)

```
In [14]: #Eliminating the 2018 data as we have only 1 month of 2018 data
```

```
df= df[df['order date (DateOrders)']<'2018-01-01']
```

```
In [15]: # Verifying new data date range
```

```
df[cols].max()
```

```
Out[15]: order date (DateOrders)      2017-12-31 23:52:00  
shipping date (DateOrders)      2018-01-06 21:45:00  
dtype: datetime64[us]
```

```
In [16]: df.shape
```

```
Out[16]: (178396, 26)
```

```
In [17]: #extracting date frm datetime columns
```

```
df[cols] = df[cols].apply(lambda x: x.dt.date)
```

```
In [18]: # Checking duplicated rows
```

```
df[df.duplicated()].shape
```

```
Out[18]: (0, 26)
```

```
In [19]: df.info()
```

```
<class 'pandas.DataFrame'>
Index: 178396 entries, 48 to 180518
Data columns (total 26 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Type                                  178396 non-null  str
1   Days for shipping (real)              178396 non-null  int64
2   Days for shipment (scheduled)         178396 non-null  int64
3   Delivery Status                       178396 non-null  str
4   Category Name                         178396 non-null  str
5   Customer City                         178396 non-null  str
6   Department Name                       178396 non-null  str
7   Market                                178396 non-null  str
8   Order City                            178396 non-null  str
9   Order Country                         178396 non-null  str
10  order date (DateOrders)               178396 non-null  object
11  Order Id                              178396 non-null  object
12  Order Item Discount Rate              178396 non-null  float64
13  Order Item Id                         178396 non-null  object
14  Order Item Product Price              178396 non-null  float64
15  Order Item Profit Ratio               178396 non-null  float64
16  Order Item Quantity                  178396 non-null  int64
17  Sales                                178396 non-null  float64
18  Order Item Total                      178396 non-null  float64
19  Order Profit Per Order                178396 non-null  float64
20  Order Region                          178396 non-null  str
21  Order Status                          178396 non-null  str
22  Product Name                          178396 non-null  str
23  Product Price                         178396 non-null  float64
24  shipping date (DateOrders)            178396 non-null  object
25  Shipping Mode                         178396 non-null  str
dtypes: float64(7), int64(3), object(4), str(12)
memory usage: 36.7+ MB
```

```
In [20]: df.describe()
```

Out[20]:

	Days for shipping (real)	Days for shipment (scheduled)	Order Item Discount Rate	Order Item Product Price	Order Item Profit Ratio	Order Q
count	178396.000000	178396.000000	178396.000000	178396.000000	178396.000000	178396.
mean	3.497578	2.932330	0.101662	141.054220	0.120744	2.
std	1.623560	1.374378	0.070418	139.742605	0.466641	1.
min	0.000000	0.000000	0.000000	9.990000	-2.750000	1.
25%	2.000000	2.000000	0.040000	50.000000	0.080000	1.
50%	3.000000	4.000000	0.090000	59.990002	0.270000	1.
75%	5.000000	4.000000	0.160000	199.990005	0.360000	3.
max	6.000000	4.000000	0.250000	1999.989990	0.500000	5.



In [21]: *# Eliminating extra spaces from column names*

```
df.columns = df.columns.str.strip()
```

EDA Part 2-

1. Investigating negative profit
2. Calculating new columns for further analysis
3. Eliminating outliers (if applicable)

In [22]: *# Investigating negative values in profit ratio and profit per order column*

```
with pd.option_context('display.max_columns', None):  
    display(df[df['Order Profit Per Order'] < 0])
```

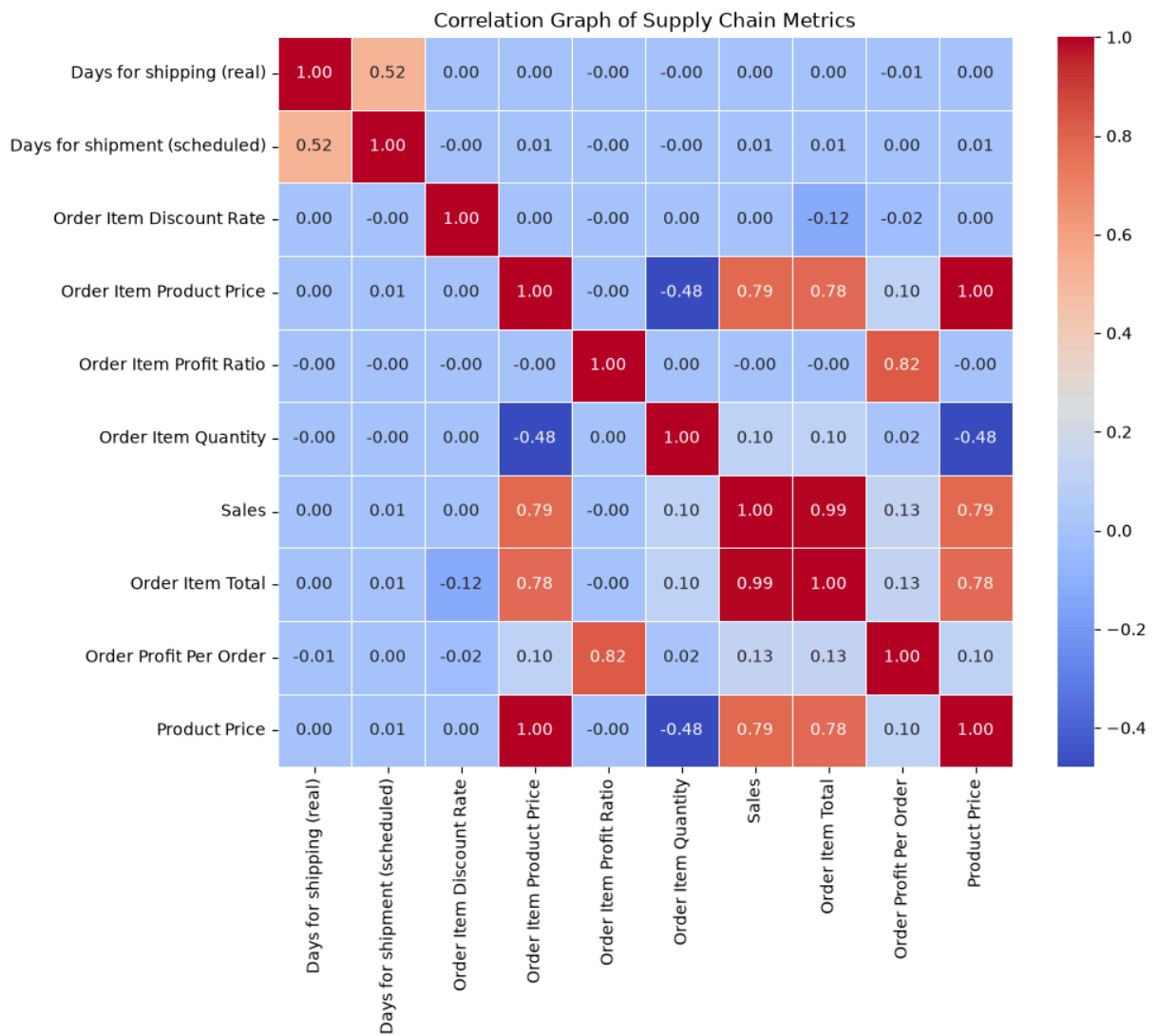
	Type	Days for shipping (real)	Days for shipment (scheduled)	Delivery Status	Category Name	Customer City	Department Name	Ma
48	PAYMENT	5	2	Late delivery	Cleats	Bayamon	Apparel	Pa
49	PAYMENT	2	2	Shipping on time	Shop By Sport	Caguas	Golf	Pa
53	PAYMENT	5	2	Late delivery	Women's Apparel	Caguas	Golf	Pa
59	PAYMENT	2	2	Shipping on time	Cardio Equipment	Caguas	Footwear	A
61	PAYMENT	5	2	Late delivery	Trade-In	Caguas	Outdoors	A
...	...	...	...	...	...	...	...	...
180495	PAYMENT	3	4	Advance shipping	Fishing	Roswell	Fan Shop	Pa
180498	CASH	3	4	Advance shipping	Fishing	Lindenhurst	Fan Shop	Pa
180499	DEBIT	4	2	Late delivery	Fishing	Caguas	Fan Shop	Pa
180506	PAYMENT	2	1	Late delivery	Fishing	San Antonio	Fan Shop	Pa

	Type	Days for shipping (real)	Days for shipment (scheduled)	Delivery Status	Category Name	Customer City	Department Name	Ma
180515	DEBIT	3	2	Late delivery	Fishing	Bakersfield	Fan Shop	Pa

33375 rows × 26 columns

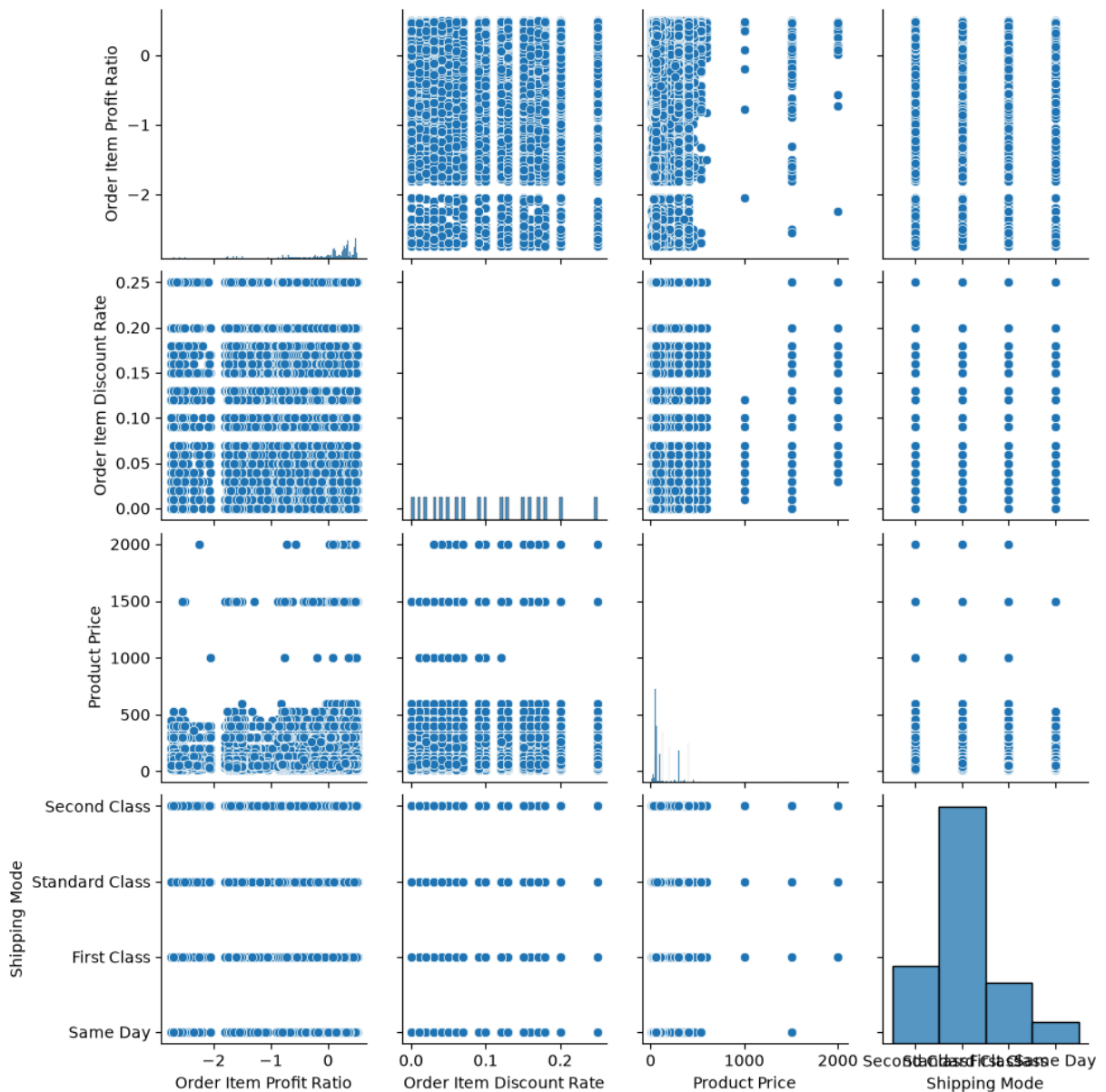
```
In [23]: # defined a correlation table to see if the negative profit margin can be explained

numeric_df = df.select_dtypes(include=['number'])
corr_matrix = numeric_df.corr()
plt.figure(figsize=(10, 8))
sns.heatmap(corr_matrix, annot=True, fmt=".2f", cmap="coolwarm", linewidths=0.5)
plt.title("Correlation Graph of Supply Chain Metrics")
plt.show()
```



```
In [24]: #sns.pairplot(data=df)
#plt.show()
```

```
In [25]: # pairplots are drawn to see if there is any non-linear relationship between any of
sns.pairplot(data=df, vars=['Order Item Profit Ratio','Order Item Discount Rate','P
plt.show()
```



Since Order Item Profit Ratio and Order Profit Per Order columns are not explained by any of our columns in given dataset we will remove them from further analysis.

```
In [26]: df=df.drop(columns=['Order Item Profit Ratio','Order Profit Per Order'])
```

```
In [27]: # calculate a new column "Shipping gap" to see by how many days scheduled delivery
# delivery days required for shipping.

df['Shipping Gap(Days)'] = df['Days for shipping (real)']-df['Days for shipment (sc
```

```
In [28]: # Verifying if the late deliveries have any shipping gap where scheduled days are m

df2 = df[df['Delivery Status']=='Late delivery'] # filtering data only for late d
df2[df2['Shipping Gap(Days)']<0]
```

Out[28]:

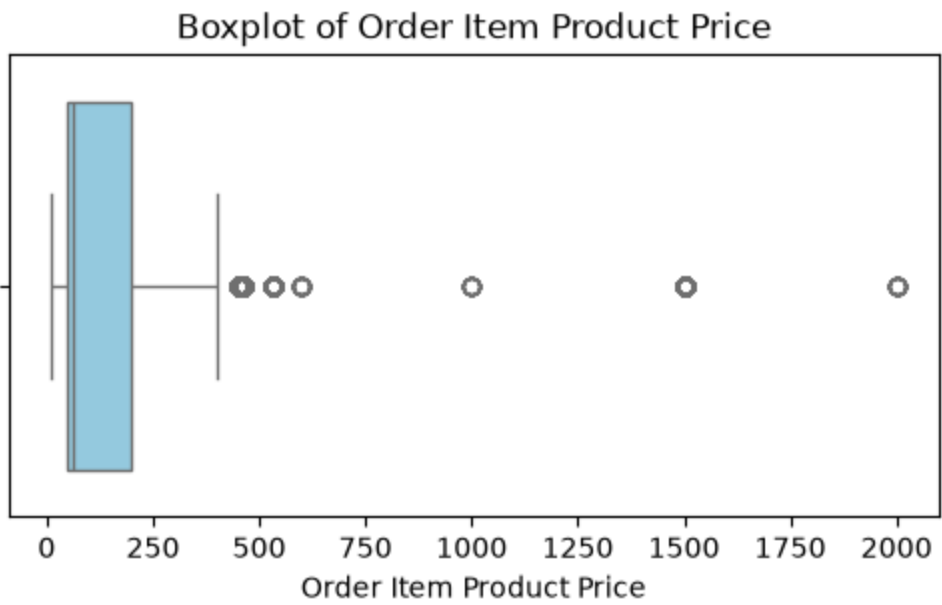
Type	Days for shipping (real)	Days for shipment (scheduled)	Delivery Status	Category Name	Customer City	Department Name	Market	Order City
------	--------------------------	-------------------------------	-----------------	---------------	---------------	-----------------	--------	------------

0 rows × 25 columns

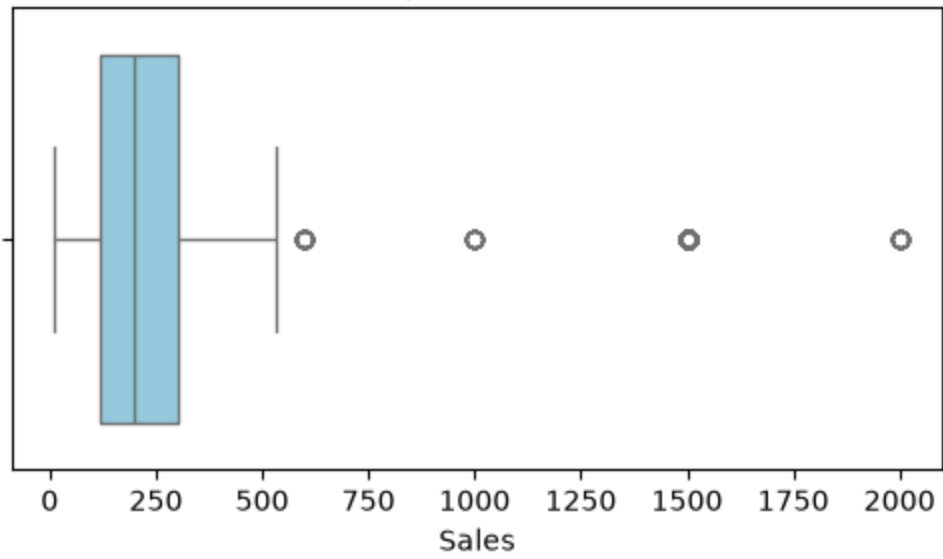


Identifying and Verifying Outlier values

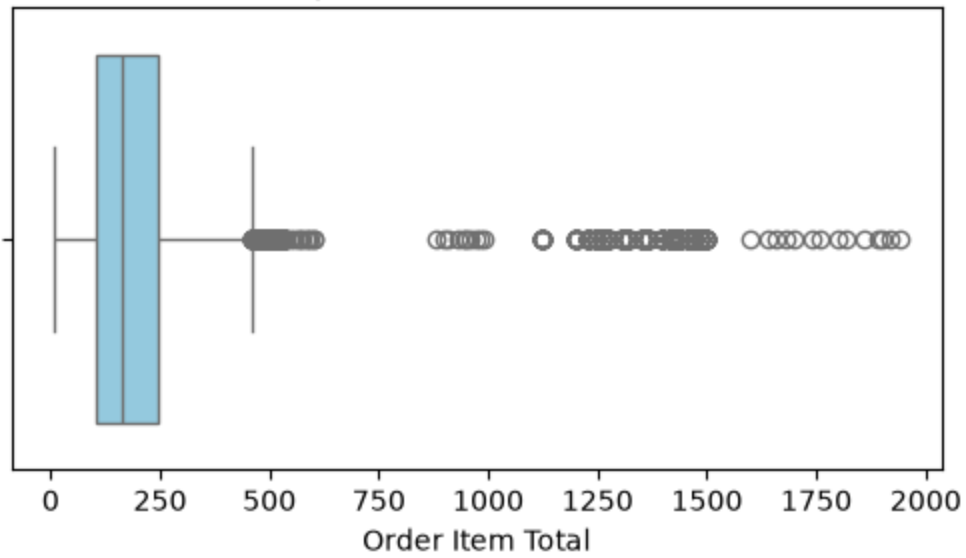
```
In [29]: cols_to_plot = ['Order Item Discount Rate', 'Order Item Product Price', 'Sales', 'Ord
for col in cols_to_plot:
    plt.figure(figsize=(6, 3))
    sns.boxplot(data=df, x=col, color='skyblue')
    plt.title(f'Boxplot of {col}')
    plt.show()
```



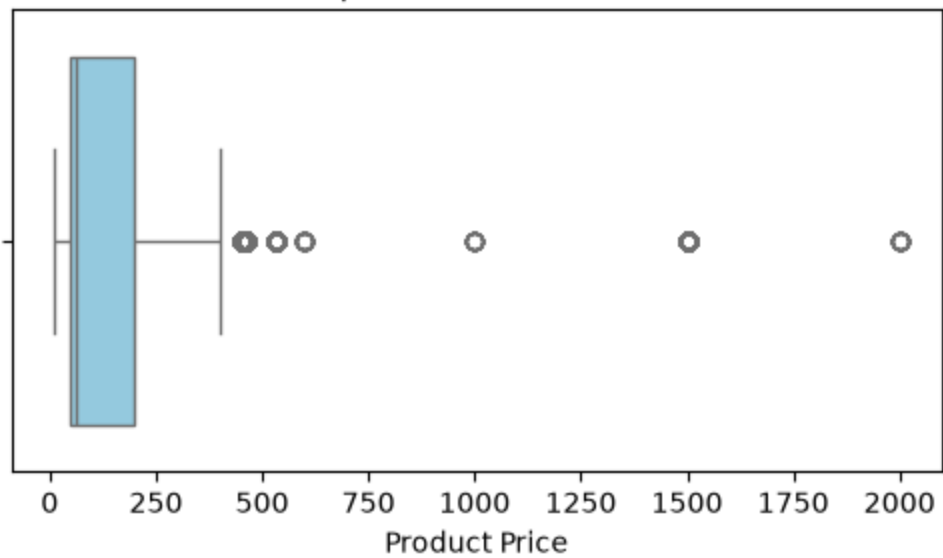
Boxplot of Sales



Boxplot of Order Item Total



Boxplot of Product Price



```
In [30]: df[df['Order Item Total']>1000]
```

Out[30]:

	Type	Days for shipping (real)	Days for shipment (scheduled)	Delivery Status	Category Name	Customer City	Department Name	Me
152	DEBIT	5	2	Late delivery	Computers	Caguas	Technology	Eu
155	DEBIT	5	2	Late delivery	Computers	Caguas	Technology	Eu
160	DEBIT	5	2	Late delivery	Computers	Caguas	Technology	Eu
172	DEBIT	5	2	Late delivery	Computers	Caguas	Technology	Eu
1353	PAYMENT	3	4	Advance shipping	Computers	Caguas	Technology	Eu
...	...	...	...	...	...	...	...	...
173436	CASH	4	4	Shipping on time	Strength Training	Caguas	Footwear	Eu
173928	CASH	5	4	Late delivery	Computers	Caguas	Technology	P
173932	CASH	6	4	Late delivery	Computers	Caguas	Technology	P
175110	DEBIT	2	1	Late delivery	Computers	Caguas	Technology	Eu
175116	DEBIT	2	1	Late delivery	Strength Training	Caguas	Footwear	Eu

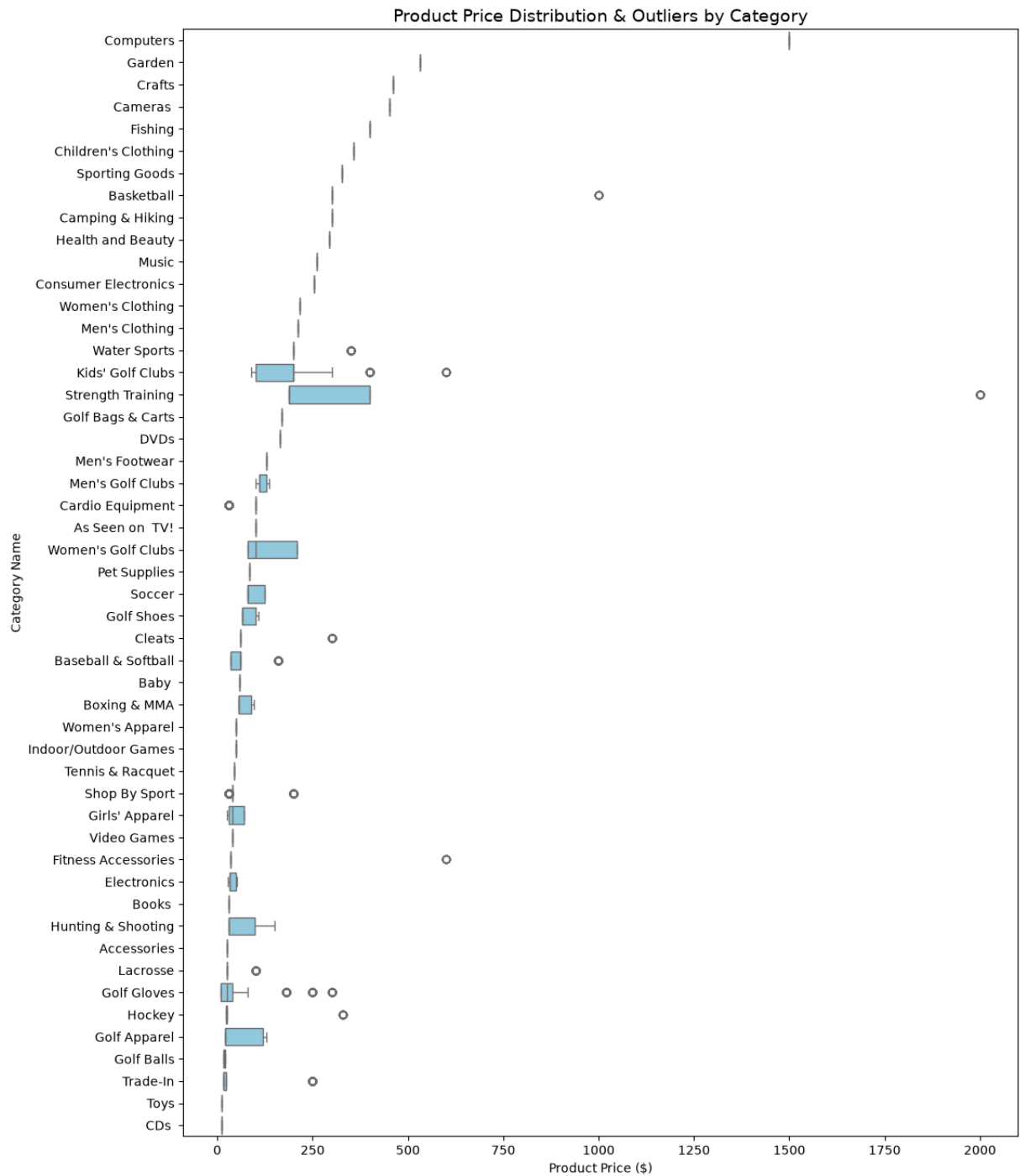
457 rows × 25 columns



```
In [31]: # Creating category wise boxplot to check outliers as per category using median price
category_order = df.groupby('Category Name')['Product Price'].median().sort_values(ascending=True)

plt.figure(figsize=(12, 16)) # Taller height so all categories fit comfortably
sns.boxplot(
    data=df,
    x='Product Price',
    y='Category Name',
    order=category_order,
    color='skyblue'
)
```

```
plt.title('Product Price Distribution & Outliers by Category', fontsize=14)
plt.xlabel('Product Price ($)')
plt.ylabel('Category Name')
plt.show()
```



```
In [32]: # Finds "Kids' Golf Club" even if you search for 'kids' or 'golf'

df2 = df[df['Delivery Status']=='Late delivery'] # filtering data only for late d
df2[df2['Category Name'].str.contains("kids", case=False) & (df2['Product Price']
```

Out[32]:

	Type	Days for shipping (real)	Days for shipment (scheduled)	Delivery Status	Category Name	Customer City	Department Name	Product Name
5836	CASH	2	1	Late delivery	Kids' Golf Clubs	Philadelphia	Outdoors	Easton
40635	DEBIT	5	4	Late delivery	Kids' Golf Clubs	Caguas	Outdoors	Easton
85138	TRANSFER	5	4	Late delivery	Kids' Golf Clubs	Mount Pleasant	Outdoors	Easton
85139	TRANSFER	5	4	Late delivery	Kids' Golf Clubs	Mount Pleasant	Outdoors	Easton
118104	TRANSFER	3	2	Late delivery	Kids' Golf Clubs	Aurora	Outdoors	Easton
122409	TRANSFER	5	2	Late delivery	Kids' Golf Clubs	Caguas	Outdoors	Easton

6 rows × 25 columns



```
In [33]: df2[(df2['Category Name']=='Basketball') & (df2['Product Price']>900) ]
```

Out[33]:

	Type	Days for shipping (real)	Days for shipment (scheduled)	Delivery Status	Category Name	Customer City	Department Name	Ma
174	DEBIT	5	2	Late delivery	Basketball	Caguas	Fitness	Eu
58483	PAYMENT	2	1	Late delivery	Basketball	Caguas	Fitness	Eu
71836	TRANSFER	5	2	Late delivery	Basketball	Brooklyn	Fitness	Eu
109710	TRANSFER	2	1	Late delivery	Basketball	Honolulu	Fitness	Eu
120715	TRANSFER	5	4	Late delivery	Basketball	Pharr	Fitness	Eu
120716	TRANSFER	6	4	Late delivery	Basketball	Los Angeles	Fitness	Eu
123529	TRANSFER	6	4	Late delivery	Basketball	Caguas	Fitness	Eu

7 rows × 25 columns



In [34]:

```
df2[(df2['Category Name']=='Strength Training') & (df2['Product Price']>1800) ]
```

Out[34]:

	Type	Days for shipping (real)	Days for shipment (scheduled)	Delivery Status	Category Name	Customer City	Department Name	Market
13343	DEBIT	5	4	Late delivery	Strength Training	Caguas	Footwear	Europe
63990	DEBIT	5	2	Late delivery	Strength Training	Frankfort	Footwear	Europe
82161	DEBIT	5	4	Late delivery	Strength Training	Augusta	Footwear	Europe
83214	DEBIT	6	4	Late delivery	Strength Training	Seattle	Footwear	Europe
141117	DEBIT	2	1	Late delivery	Strength Training	Caguas	Footwear	Europe
175116	DEBIT	2	1	Late delivery	Strength Training	Caguas	Footwear	Europe

6 rows × 25 columns



There are no actual outliers in the data. Certain high end products have higher price.

Checking Order Status For Late Deliveries.

```
In [35]: df['Order Status'].unique()
```

```
Out[35]: <StringArray>
['PENDING_PAYMENT', 'PENDING', 'PROCESSING', 'ON_HOLD',
 'COMPLETE', 'SUSPECTED_FRAUD', 'CANCELED', 'PAYMENT_REVIEW',
 'CLOSED']
Length: 9, dtype: str
```

```
In [36]: df['Delivery Status'].unique()
```

```
Out[36]: <StringArray>
['Late delivery', 'Shipping on time', 'Advance shipping', 'Shipping canceled']
Length: 4, dtype: str
```

```
In [37]: Cancelled_shipping=df[df['Delivery Status']=='Late delivery']
Cancelled_shipping['Order Status'].unique()
```

```
Out[37]: <StringArray>
['PENDING_PAYMENT',          'PENDING',          'PROCESSING',          'ON_HOLD',
      'COMPLETE', 'PAYMENT_REVIEW',          'CLOSED']
Length: 7, dtype: str
```

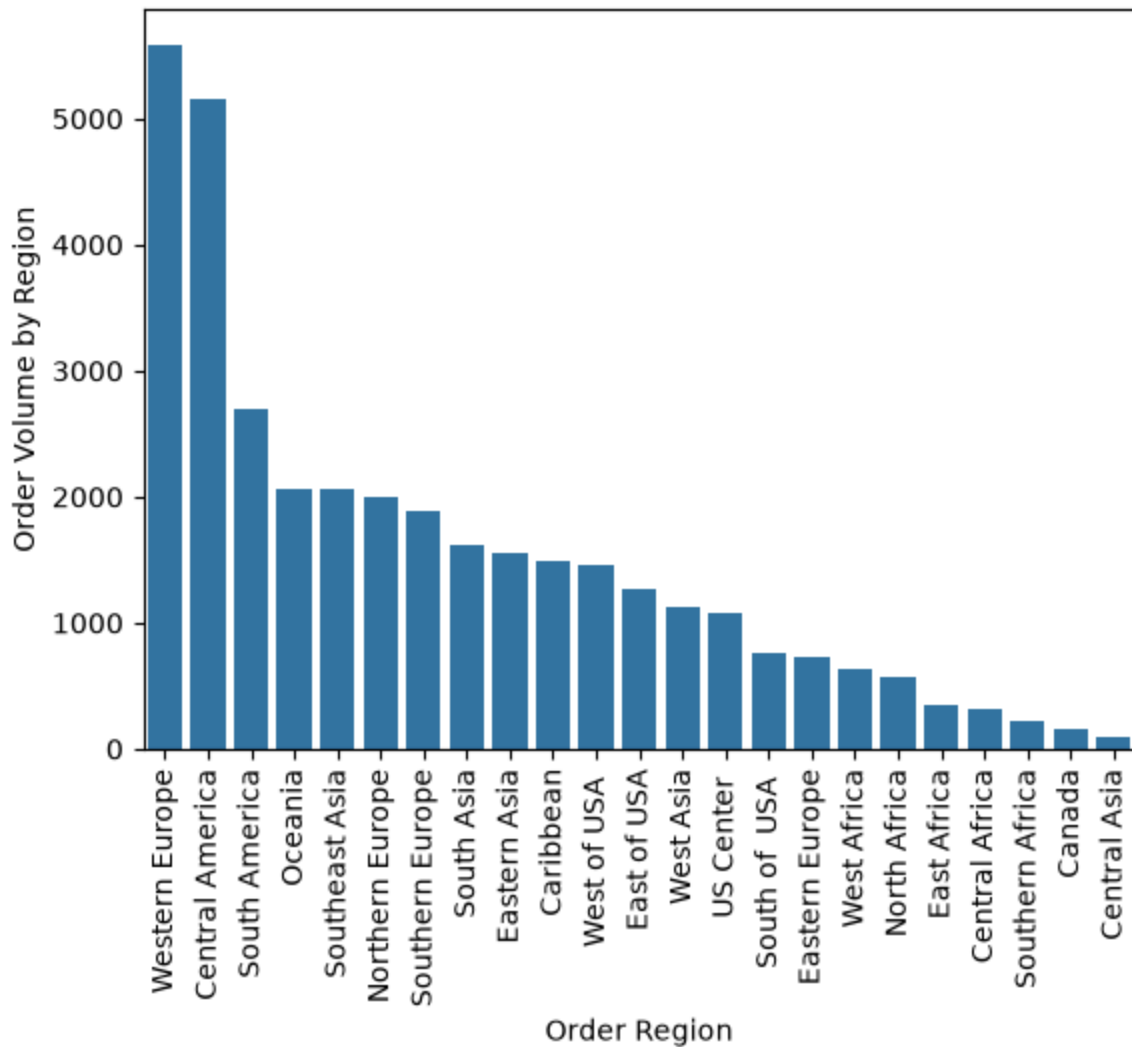
Plotting data of late deliveries by region and by Sales risk.

```
In [38]: df['Order Region'].nunique()
```

```
Out[38]: 23
```

```
In [40]: #Volume of late deliveries in each region
```

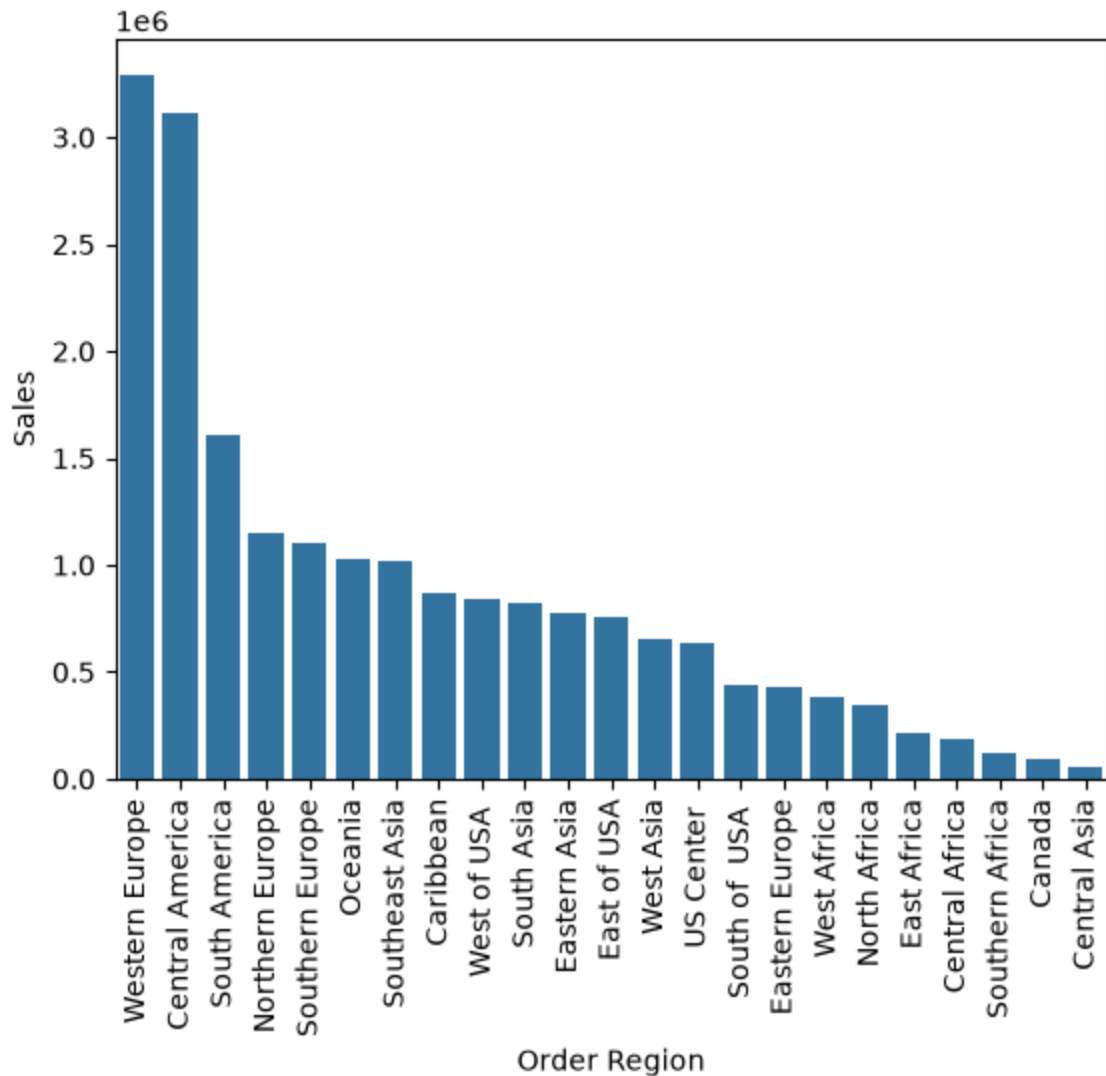
```
late_order_volume_by_region = df[df['Delivery Status']=='Late delivery'].groupby('O
sns.barplot(x=late_order_volume_by_region.index, y=late_order_volume_by_region.valu
plt.xticks(rotation=90)
plt.ylabel('Order Volume by Region')
plt.show()
```



```
In [42]: # Plotting risk to sales in regions due to late deliveries
```

```
late_deliveries =df[df['Delivery Status']=='Late delivery']
```

```
sales_order= late_deliveries.groupby('Order Region')['Sales'].sum().sort_values(asc
sns.barplot(data=late_deliveries, x='Order Region', y='Sales', estimator = sum, erro
plt.xticks(rotation=90)
plt.show()
```



It is evident that Western Europe, Central America and South America are the top 3 regions that needs to be prioritized due to higher order volume and Sales (\$).

```
df.to_csv(r'C:\Users\shobh\Downloads\SupplyChainDataset\supply_chain_clean.csv',
index=False)
```